

Kunming AI+

An Applied Introduction to AI with PyTorch

Luca Ghafourpour



**UNIVERSITY OF
CAMBRIDGE**
Department of Engineering

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance + <i>Weights and Biases</i>
Practical Workshop 3	Initialisation and regularisation
Academic Lecture 1	Physics Informed Neural Networks
Academic Lecture 2	Neural Operators
Workshop 4	Convolutional neural networks
Practical Workshop 4	Scikit Learn
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

Two ways to ensure poor performance with deep networks

Starting

Before updating parameters, the weights are chosen to be random numbers.

If we pick the wrong scale, the signal may vanish or explode as it flows through the layers, leading to a model that learns slowly, or not at all.

Fix: Initialisation

Generalising

After fitting the training data, the model must still perform well on new examples.

If the network is too flexible, it may memorise the training set rather than learn a pattern that transfers, so training loss falls while validation loss rises.

Fix: Regularisation

Initialisation

Why initialisation matters

Each neural network layer multiplies a signal by a factor near 1. Over many layers, small errors compound.

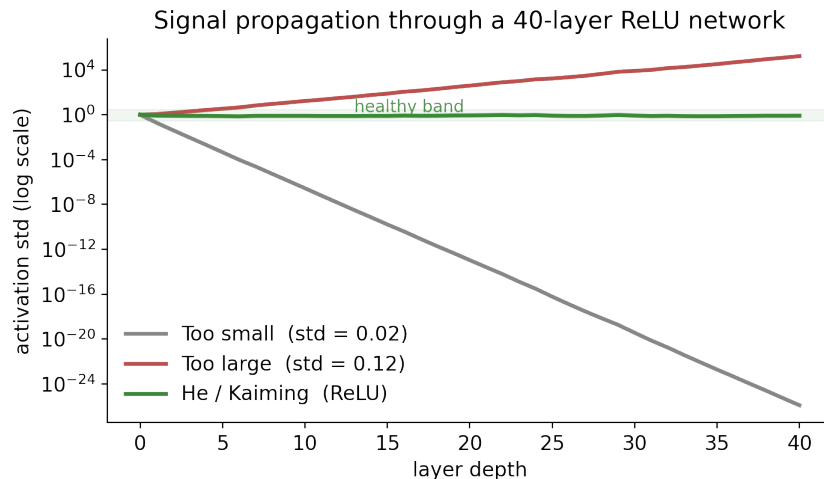
Vanishing Gradients

If each layer shrinks the activations a little, after 40 layers they are ≈ 0 .

Exploding Gradients

If each layer grows them a little, they diverge.

The same happens to gradients on the backward pass. The network either stops learning or diverges to NaN.



The right initialisation keeps the variance of activations (and gradients) roughly constant with depth.

What a good initialisation should do

1. Break symmetry

Imagine if every weight is initialised to the same value, and every neuron in a layer computes the same thing and gets the same gradient.

Weights stay identical forever.

Fix: weights must be random.

2. Preserve scale

But! Being random is not enough.

Key: the variance must be right. Too large saturates / explodes; too small vanishes. The scale is set by the layer width and the activation.

The variance argument

Here $\mathbf{n}$ = inputs to the layer (fan-in), $\mathbf{m}$ = outputs (fan-out).

Consider the pre-activation $z = Wx$, where $x \in \mathbb{R}^n$ has independent, zero-mean entries with variance γ^2 . Each of the m outputs is a sum of n independent terms, so its variance is $n \cdot \text{Var}(W) \cdot \gamma^2$. Setting $\text{Var}(W) = 1/n$ cancels it and keeps the variance stable through depth.

Forward — keep the signal

$$z = Wx$$

$$\text{Var}(z) = \mathbf{n} \cdot \text{Var}(W) \cdot \gamma^2$$

$$\Rightarrow \text{Var}(W) = 1 / n$$

Backward — keep the gradient

$$\partial L / \partial x = W^T (\partial L / \partial z)$$

$$\text{Var}(\partial L / \partial x) = \mathbf{m} \cdot \text{Var}(W) \cdot \text{Var}(\partial L / \partial z)$$

$$\Rightarrow \text{Var}(W) = 1 / m$$

LeCun initialisation

$$w_{ij} \sim \mathcal{N}(0, \sigma^2)$$

$$\sigma^2 = \frac{1}{n}, \quad b_i = 0$$

Best for: SELU activation functions and linear layers

Enforces only the forward condition: divides variance by n , so activations pass through unchanged.

Assumes the activation is $\approx$ linear near 0 (variance-transparent): true for tanh, less so for sigmoid.

Xavier / Glorot initialisation

$$\sigma^2 = \frac{2}{n + m}$$

$$w_{ij} \sim \text{Uniform}[-\sqrt{3} \sigma, \sqrt{3} \sigma]$$

Forward condition wants $\text{Var}(W)=1/n$,
backward wants $\text{Var}(W)=1/m$.

Both cannot hold at once so Xavier/Glorot
compromises with their average.

Derived for activations that are symmetric
with unit slope at 0 (tanh), so the variance
carries cleanly through both passes.

Best for: tanh / sigmoid networks, or
general symmetric activations

He / Kaiming initialisation

$$\sigma^2 = \frac{2}{n}$$

$$w_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Best for: ReLU, LeakyReLU, GELU.

ReLU zeros its negative half ($z < 0$), roughly halving the post-activation variance at every layer.

Double the weight variance ($2/n$, not $1/n$) to cancel that $\frac{1}{2}$ and hold the variance constant through depth.

Without the factor of 2 a deep ReLU net's signal decays geometrically.

Pytorch implementation

```
import torch.nn as nn

initialization == # Add initialization as a string

if initialization == "lecun"
    nn.init.normal_(model.weight, std=(1.0 / model.weight.size(1)) ** 0.5)

if initialization == "kaiming":
    nn.init.kaiming_normal_(model.weight, nonlinearity='relu')

if initialization == "xavier":
    nn.init.xavier_uniform_(model.weight,
gain=nn.init.calculate_gain('tanh'))

nn.init.zeros_(model.bias)
```

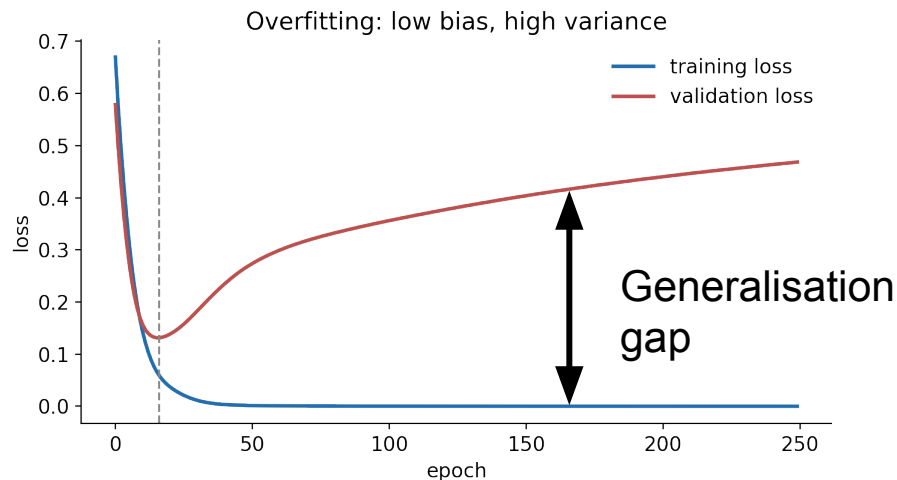
Regularisation

What is regularisation?

Last lecture, we saw that overfitting resulted in high variance.

The model has enough capacity to memorise the training set, so train loss keeps falling while validation loss turns and rises.

Regularisation accepts slightly higher bias in exchange for lower variance shrinking the **generalisation gap** between the train and validation curves.



Weight decay

Add an L2 penalty to the loss:

$$\tilde{L}(\theta) = L(\theta) + \frac{\lambda}{2} \|w\|_2^2$$

which makes every step shrink the weight:

$$w \leftarrow (1 - \eta\lambda) w - \eta \nabla_w L$$

Typical λ : 1e-4 to 1e-2.

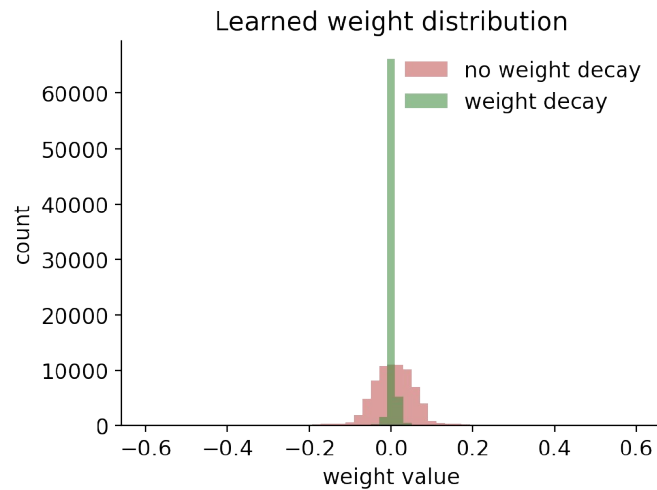
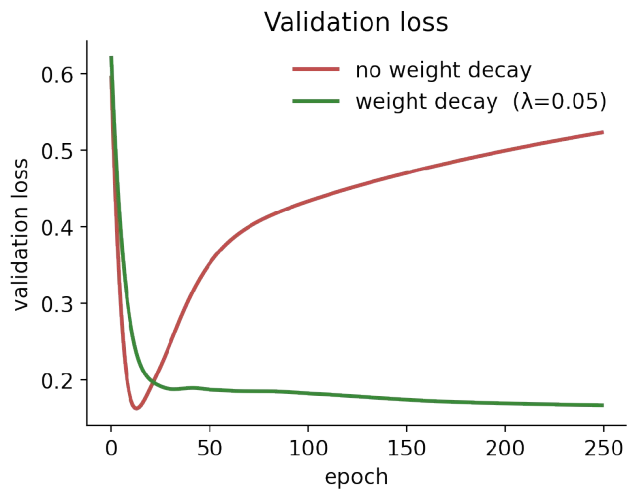
Penalises large weights, resulting in smoother, simpler functions.

The $(1 - \eta\lambda)$ factor pulls weights toward 0 each step.

Smaller weights yield functions with lower-curvature function, and hence less overfitting.

L2 and weight decay coincide for SGD but not for Adam. For Adam, use AdamW variant.

Weight decay



Batch normalisation

Idea: standardise each feature across the batch, then let the network rescale it.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \qquad y = \gamma \hat{x} + \beta$$

Steps:

1. Standardise: rescale each feature to mean 0, variance 1 using this batch's mean and variance.
2. Rescale: learned γ , β (per feature) lets the network pick the distribution it wants.
3. At eval: use running averages instead of batch stats, so predictions are stable.

- ✓ Smooths the loss landscape, allowing deeper nets and higher learning rates.
- ✓ Regularises mildly, since the batch-dependent noise acts like a gentle dropout.

Layer normalisation

Idea: standardise each sample across its own features, then let the network rescale it.

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} \qquad y = \gamma \hat{x}_i + \beta$$

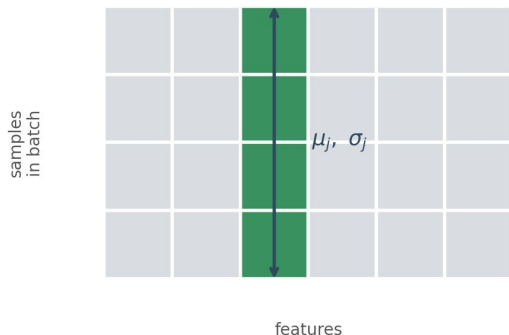
Steps:

1. Standardise: rescale each sample to mean 0, variance 1 using that sample's own features.
 2. Rescale: learned γ , β (per feature) let the network pick the distribution it wants.
 3. Same at train and eval: nothing is measured from the batch, so it works at batch size 1 and needs no running averages.
- ✓ Independent of batch size, so works on batch size 1 and variable length sequences
 - ✓ The default in Transformers & Recurrent Neural Networks.

Batch vs Layer normalisation

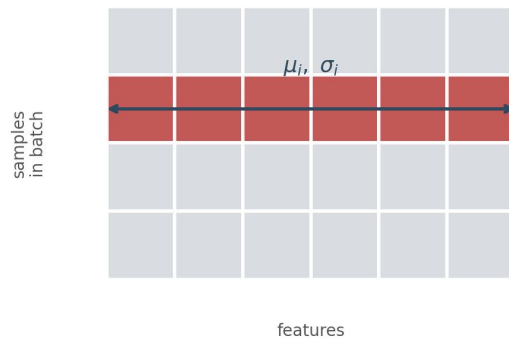
Batch Norm

average over the batch (one stat per feature)



Layer Norm

average over the features (one stat per sample)



	Batch Norm	Layer Norm
Batch-dependent?	yes	no
Regularises?	mildly (batch noise)	no
Train vs eval	differs (running stats)	identical
Typical use	CNNs, large batches	Transformers, RNNs

Dropout

Idea: Randomly switch off activations during training so no neuron can rely on another.

$$\tilde{h} = \frac{m \odot h}{1 - p}, \quad m_i \sim \text{Bernoulli}(1 - p)$$

Each forward pass, zero each hidden unit independently with probability p .

Divide survivors by $(1 - p)$ so the expected activation is unchanged.

At test time the full network is used, no rescaling needed.

Forces redundant, distributed representations instead of fragile co-adaptations.

Typical p

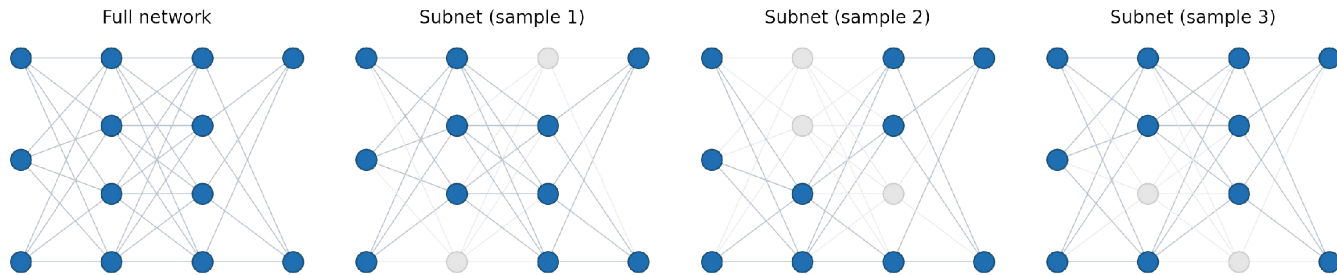
0.1 – 0.3 for hidden layers; up to 0.5 for large dense layers.

Remember `model.eval()`

or dropout stays on at test time and your predictions become noisy.

Dropout

Dropout trains a different thinned subnetwork on every step — test time averages them



A net with n units contains 2^n possible thinned sub-networks. Dropout trains a different one on every step, all sharing the same weights.

At test time, use all units at once as a fast approximation to averaging the whole ensemble!

$$y \approx \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} f(x; m \odot \theta)$$

approximate model averaging

Early stopping

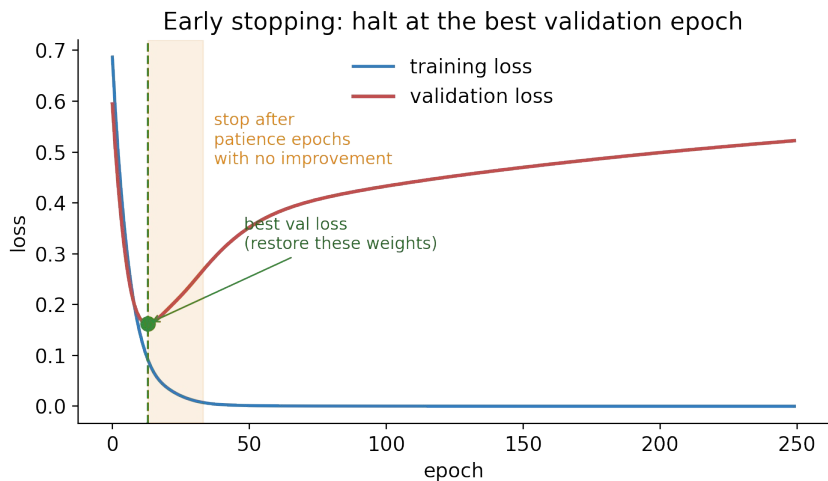
Idea: The cheapest regulariser: stop before the model starts to overfit.

In workshop 3 we saw that before a model began to overfit, the loss curves would look relatively healthy.

Idea: track validation loss every epoch, and stop the training when the validation loss begins to increase over a *patience* window.

It implicitly limits how far the weights travel from their initial values

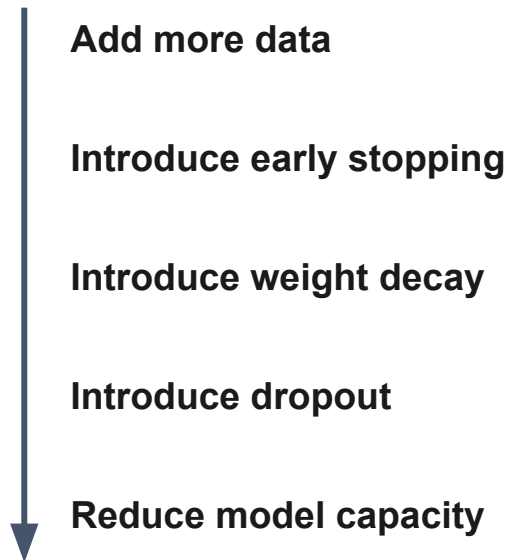
Very popular form of regularisation that costs nothing.



Regularisation summary

Method	Mechanism	Main knob	Notes
Weight decay	shrinks weights toward 0	λ	use AdamW with Adam
Batch Norm	normalise over the batch	—	higher LR; eval $\neq$ train
Layer Norm	normalise over features	—	batch-independent; transformers
Dropout	random unit masking (ensemble)	p	off at eval()
Early stopping	halt at best validation epoch	patience	free; restore best weights

A practical workflow for fixing overfitting



Time for some code...